Харківський національний університет радіоелектроніки

Факультет комп’ютерних наук

Кафедра програмної інженерії

ЗВІТ

до лабораторної роботи №1

з дисципліни «Технології розробки гібридних систем штучного інтелекту»

на тему «РОЗРОБЛЕННЯ RAG-СИСТЕМИ. ПРОЕКТУВАННЯ ПЗ»

Виконала:
ст. гр. ІПЗм-26-1,
Каленик Віра Олександрівна

Перевірив:
ст. викладач каф. ПІ,
Дашенков Дмитро Сергійович

Харків 2026

1.1 Мета роботи

Метою лабораторної роботи є набуття практичних навичок попереднього проєктування програмного забезпечення для RAG-системи.

1.2 Хід виконання

1.2.1 Сформулювати сценарій використання RAG-системи обсягом приблизно одна сторінка. Опис повинен охоплювати: цільового користувача; типове запитання, яке користувач задасть системі; критерії гарної та поганої відповіді. Обраний сценарій має бути достатньо конкретним для подальшого оцінювання якості системи.

Темою системи було обрано створення асистента для нових контриб'юторів Kubernetes, який буде надавати відповіді на питання про правила участі в спільноті на основі документації.

Kubernetes — це платформа з відкритим вихідним кодом, створена для автоматизації розгортання, масштабування та керування застосунками у контейнерах [1]. Оскільки код відкрито, тисячі людей з різних компаній роблять свій внесок у розвиток продукту. Правила участі зібрані в сотнях документів репозиторію [2]. Тут описано як створити pull request, як проходить рев'ю, як стати членом спільноти і до якої робочої групи звернутися. Тож, щоб новачки не витрачали багато часу на пошук потрібної інформації і очікування відповіді в Slack, RAG-асистент буде відповідати на питання за документацією спільноти і посилатися на відповідні документи.

Цільовим користувачем системи буде розробник, який готується зробити перший внесок у Kubernetes або щойно його зробив, володіє англійською, знає git та GitHub, але не знає внутрішніх процесів і структури спільноти.

Типовими запитаннями можуть бути:

- How do I find a good first issue to work on?
- What do the lgtm and approved labels mean?
- How do I become a Kubernetes org member?
- Which SIG is responsible for networking?
- My PR has been open for two weeks, who should I ping?

- How do I report a Code of Conduct violation?

Відповідь буде вважатися хорошою, якщо буде спиратися лише на знайдені уривки документації, міститиме посилання на них, надаватиме конкретні дії для вирішення проблеми (наприклад, яку команду написати в коментарі, куди звернутися) і якщо інформацію буде не знайдено, прямо повідомить про це та порадить звернутися до спільноти через Slack або пошту.

Відповідь буде вважатися поганою, якщо буде містити посилання на невідомі джерела або не міститиме посилань, повідомлятиме вигадані правила, команди чи вимоги, яких немає в документації, надаватиме загальну інформацію або таку, що не є відповіддю на задане питання та якщо буде виконувати команди, вбудовані в запит, всупереч системним (prompt injection) [3].

Наприклад, якщо користувач задає системі питання «How do I become a Kubernetes org member?», то хорошою відповіддю буде вважатися наступна:

```
«To become a member of the Kubernetes GitHub organization you need to:
1) enable two-factor authentication on GitHub and keep your company affiliation up to date [1];
2) make multiple contributions over time, including at least one merged PR [1];
3) subscribe to dev@kubernetes.io and read the contributor guide [1];
4) get sponsorship from 2 reviewers who have worked closely with you; they must be from different companies [1].
Then open a membership request issue in the kubernetes/org repo, mention your sponsors and fill in the checklist. The sponsors confirm with +1 [1].
Sources: [1] community-membership.md, section «Member»»,
```

оскільки вона ґрунтується на конкретному джерелі, містить посилання на джерело тверджень та конкретні дії для вирішення проблеми [4]. Поганою відповіддю була б така, що містила б галюцинації, як наприклад, «You need at least 5 merged PRs and must pass a short online exam», правильний зміст, але з хибним посиланням, як наприклад, «Sources: [12] community-membership.md, section «Member»», або надала б відповідь на інше питання, що не стосується

теми, як наприклад, «To install kubectl you need …». Також поганою відповіддю вважається така, що містить правильну, але неповну відповідь на задане питання, для наданого прикладу це була б відповідь, в якій згадано про 2FA та merged PR, але не зазначено про спонсорів.

1.2.2 Обрати корпус даних обсягом від п'ятисот до десяти тисяч документів з варіантів, перелічених у Вступі або знайти власний аналог. У звіті задокументувати: назву та джерело набору даних, ліцензію, оцінку обсягу (кількість документів).

Корпусом даних для системи було обрано згадану вище документацію спільноти Kubernetes, яка зберігається в репозиторії kubernetes/community на GitHub. Репозиторій поширюється за ліцензією Apache-2.0, тобто можна вільно використовувати та поширювати матеріали за умови згадки про авторство та ліцензію [5].

На момент написання лабораторної роботи репозиторій містить 976 документів, з яких 905 знаходяться поза папкою archive (застарілі матеріали). До робочого корпусу буде включено посібник контриб'ютора, сторінки робочих груп (SIG), правила членства та управління проєктом, канали комунікації й менторство. Папки elections (181 документ) та events (99 документів) буде виключено, оскільки вони містять переважно біографії кандидатів на виборах та описи подій. Отже, орієнтовний обсяг корпусу становитиме близько 620 документів, що вкладається в межі від п'ятисот до десяти тисяч.

1.2.3 Зафіксувати технологічний стек проєкту. Для кожного з чотирьох ключових рішень — векторне сховище, модель ембедингів, модель генерації, мова програмування — навести вибір та коротке (одне-три речення) обґрунтування. Рекомендується використання лише безкоштовних інструментів з відкритим кодом або безкоштовних рівнів використання сервісів.

Щодо технологічного стеку було обрано векторне сховище Chroma, модель ембедингів all-MiniLM-L6-v2 з бібліотеки sentence-transformers, модель

генерації Qwen 2.5 3B, яка запускатиметься локально через Ollama, та мову програмування Python [6-10].

Chroma працює як звичайна Python-бібліотека, тому не потрібно буде піднімати окремий сервер чи Docker. Вектори зберігатимуться разом з метаданими чанків.

Потужної відеокарти в ноутбуці немає (NVIDIA GeForce MX330 на 2 ГБ пам'яті), тому ембедінги рахуватимуться на процесорі. Модель all-MiniLM-L6-v2 компактна, тож має закодувати весь корпус за кілька хвилин.

Розмір моделі генерації було обрано через швидкість. Модель на 7 млрд параметрів на ноутбуці вже запускалася, і обробка 10 прикладів зайняла близько 27 хвилин. Для тестування потрібно буде прогнати десятки запитів, тому основною стала модель на 3 млрд параметрів.

Python обрано тому, що всі потрібні бібліотеки (chromadb, sentence-transformers, ollama, rank_bm25) мають саме Python-інтерфейс, а для обробки тексту й скриптів оцінювання він найзручніший.

1.2.4 Побудувати спрощену контекстну діаграму архітектури системи у будь-якому інструменті для діаграм (draw.io, Excalidraw, Mermaid тощо). Формат діаграми можна обрати на свій розсуд: довільну блок-схему, UML діаграму компонентів чи послідовностей тощо. На діаграмі повинні бути ідентифіковані основні компоненти: пайплайн прийому даних; векторний індекс; ретривер; модель генерації; інтерфейс користувача.

Для представлення архітектури системи було зроблено дві UML-діаграми, побудовані за допомогою Excalidraw [11].

Перша з них є діаграмою послідовностей (рис. 1) і показує, з яких компонентів складається система та в якому порядку вони взаємодіють. Діаграма складається з двох частин: індексування та обробки запиту користувача. Під час індексування пайплайн прийому даних очищує документи, розбиває їх на чанки та зберігає у векторний індекс Chroma і BM25-індекс. Під час обробки запит через CLI потрапляє до ретривера, який шукає релевантні чанки двома способами (за вектором і за словами) та об'єднує результати методом RRF. Знайдені п'ять чанків разом із питанням передаються моделі генерації, а її

відповідь перевіряє валідатор цитат. Якщо цитати коректні, користувач отримує відповідь із джерелами. Якщо ні, система один раз повторює запит до моделі, а за повторної помилки попереджає користувача, що не впевнена у відповіді. Так модель не може передати користувачеві відповідь з неперевіреними посиланнями.

Друга є діаграмою розгортання (рис. 2) і показує, де саме працюють компоненти системи. Усе розміщено локально на ноутбуці, але в трьох окремих середовищах. У середовищі Python 3.11 працюють CLI, пайплайн прийому даних, модуль ембедингів, ретривер і валідатор цитат. Векторний індекс Chroma та BM25-індекс зберігаються як файли на диску, а модель генерації запускається процесом Ollama, з яким Python спілкується через HTTP на порту 11434. Ззовні система звертається лише до GitHub, щоб завантажити документи репозиторію kubernetes/community, тож після індексування може працювати без доступу до інтернету.

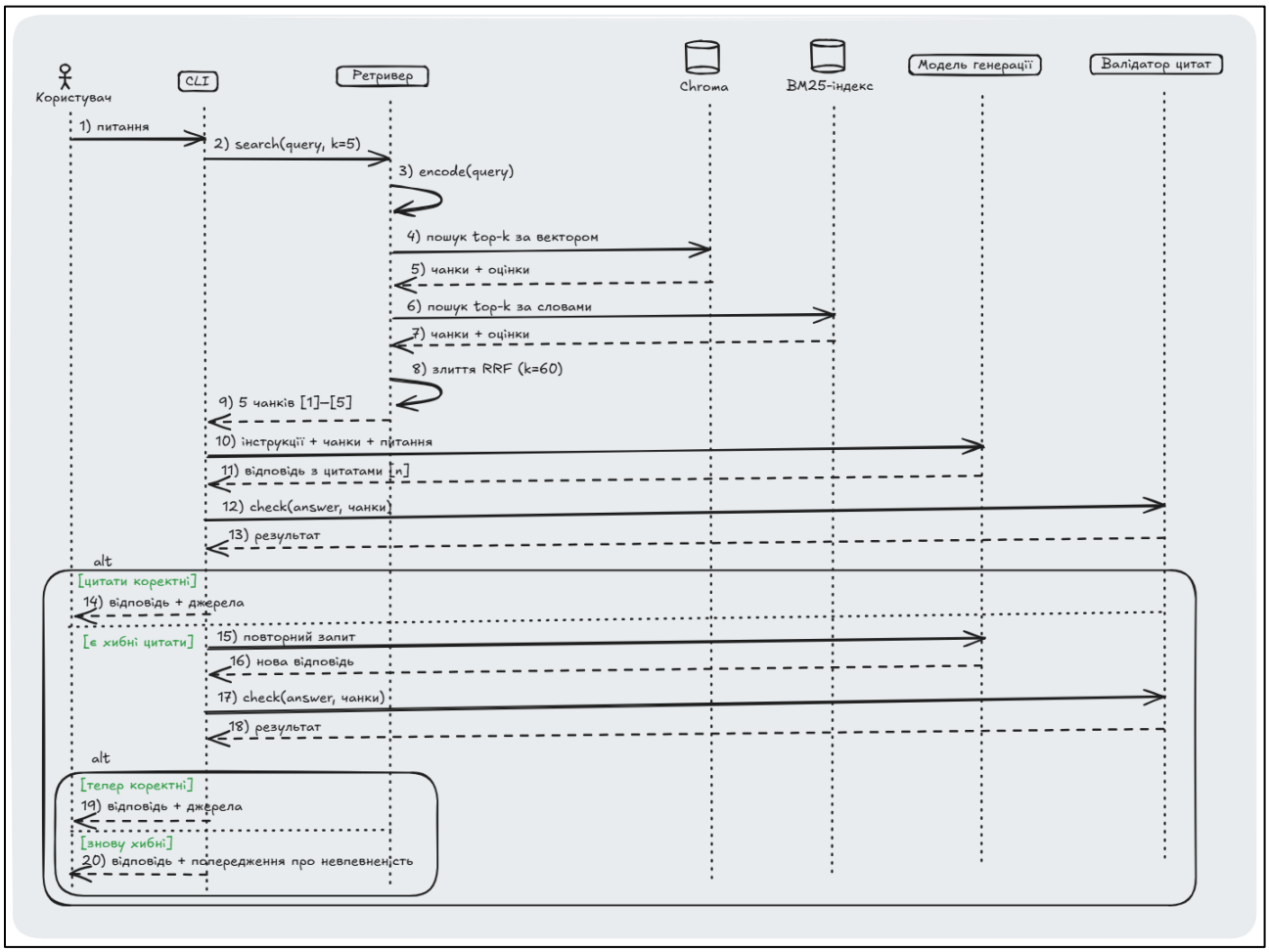


Рисунок 1 — UML-діаграма послідовностей

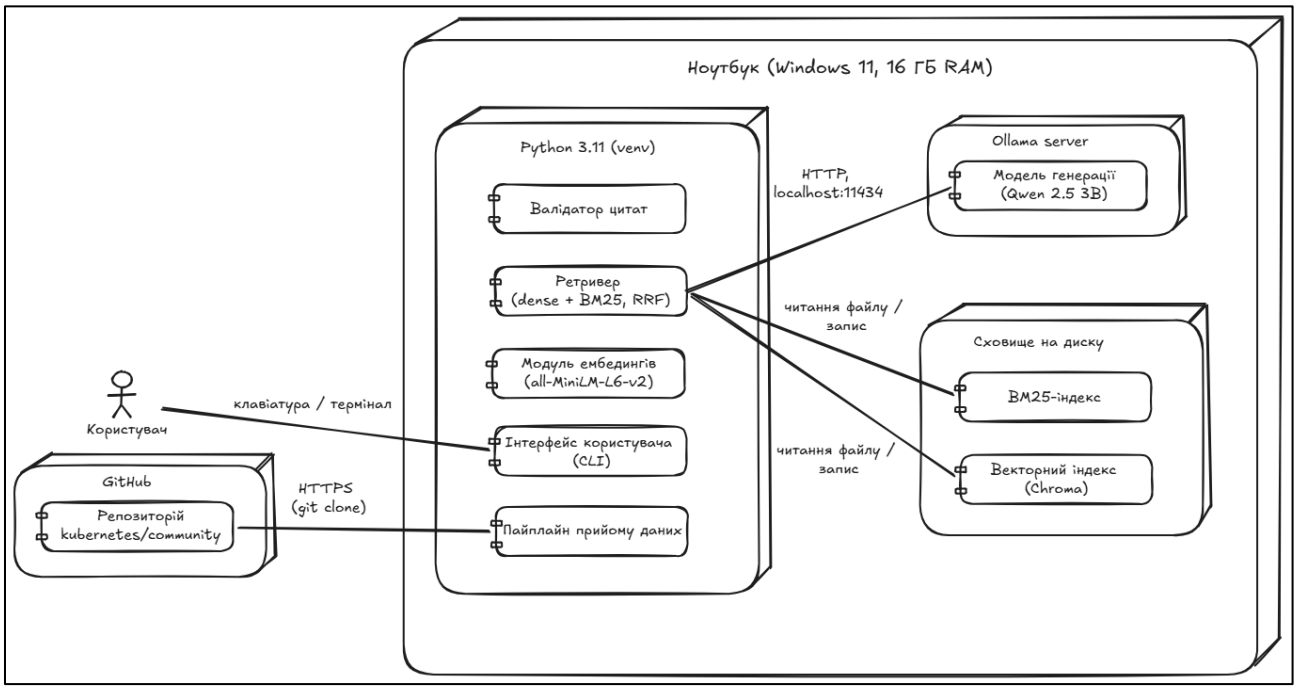


Рисунок 2 — UML-діаграма розгортання

1.2.5 Ініціалізувати Git-репозиторій із базовою структурою каталогів src/, config/, reports/. Розмістити у репозиторії файл README.md, що містить ім'я проєкту, стислий опис обраного сценарію та інструкції із встановлення обраного стеку. Створити тег lab-1 та надіслати його на GitHub.

Для проєкту було створено публічний репозиторій rag-project-kalenyk на GitHub [12]. Він містить три базові каталоги: src/ для програмного коду, config/ для налаштувань (шляхи, назви моделей, параметри чанкування) та reports/ для звітів і діаграм.

У файлі README.md наведено назву проєкту, стислий опис сценарію та інструкції з встановлення стеку. Після завершення роботи було створено тег lab-1 і надіслано його на GitHub.

1.3 Висновки

У ході виконання лабораторної роботи було спроєктовано RAG-асистента для нових контриб'юторів Kubernetes, а саме сформульовано сценарій використання, обрано корпус і локальний стек, побудовано дві UML-діаграми та створено репозиторій.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Kubernetes Overview. URL: https://kubernetes.io/docs/concepts/overview/;

2. Kubernetes Community Documentation. URL: https://github.com/kubernetes/community;

3. OWASP. LLM01: Prompt Injection. URL: https://genai.owasp.org/llmrisk/llm01-prompt-injection/;

4. Kubernetes Community Membership. URL: https://github.com/kubernetes/community/blob/main/community-membership.md;

5. Apache License, Version 2.0. URL: https://www.apache.org/licenses/LICENSE-2.0;

6. Chroma Docs. URL: https://docs.trychroma.com/;

7. Sentence Transformers Documentation. URL: https://sbert.net/;

8. all-MiniLM-L6-v2. URL: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2;

9. Qwen 2.5. URL: https://ollama.com/library/qwen2.5;

10. Ollama. URL: https://ollama.com/;

11. Excalidraw. URL: https://excalidraw.com/;

12. RAG-project-kalenyk. URL: https://github.com/NureKalenykVira/rag-project-kalenyk/tree/lab-1.

ДОДАТОК А

Декларація щодо використання штучного інтелекту

Під час виконання роботи використовувався асистент Claude (Anthropic), модель Claude Opus 5.5, у вебзастосунку claude.ai.

ШІ використовувався для обговорення вибору теми, корпусу даних і технологічного стеку, перевірки ліцензії та підрахунку кількості документів у репозиторії kubernetes/community, а також для підготовки тексту README.md.

Отримані результати я перевіряла: ліцензію та структуру репозиторію на GitHub [2], вимоги до членства у файлі community-membership.md [4], характеристики ноутбука у Диспетчері завдань. Під час перевірки було виправлено неточності, зокрема у формулюванні щодо часу роботи моделі.